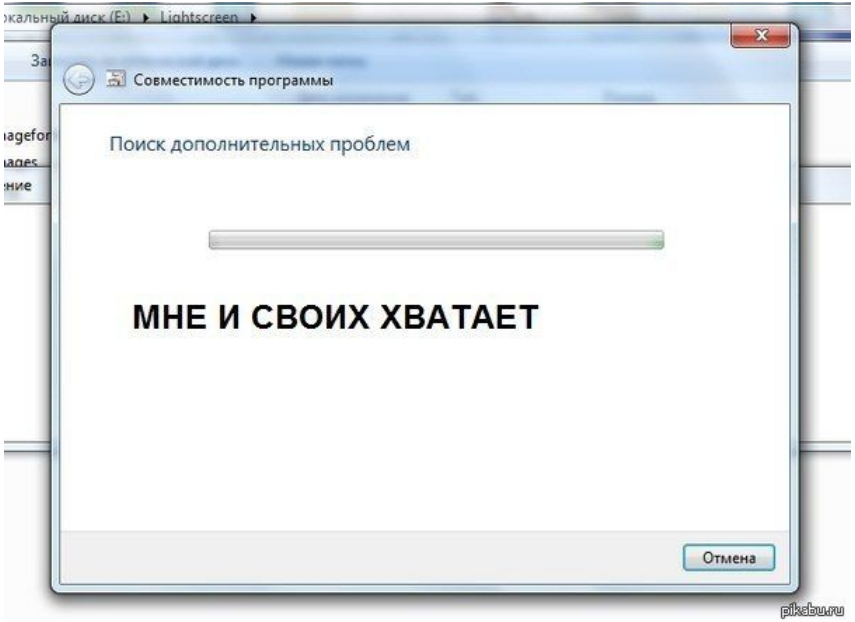
FAST FORWARD

АКОС

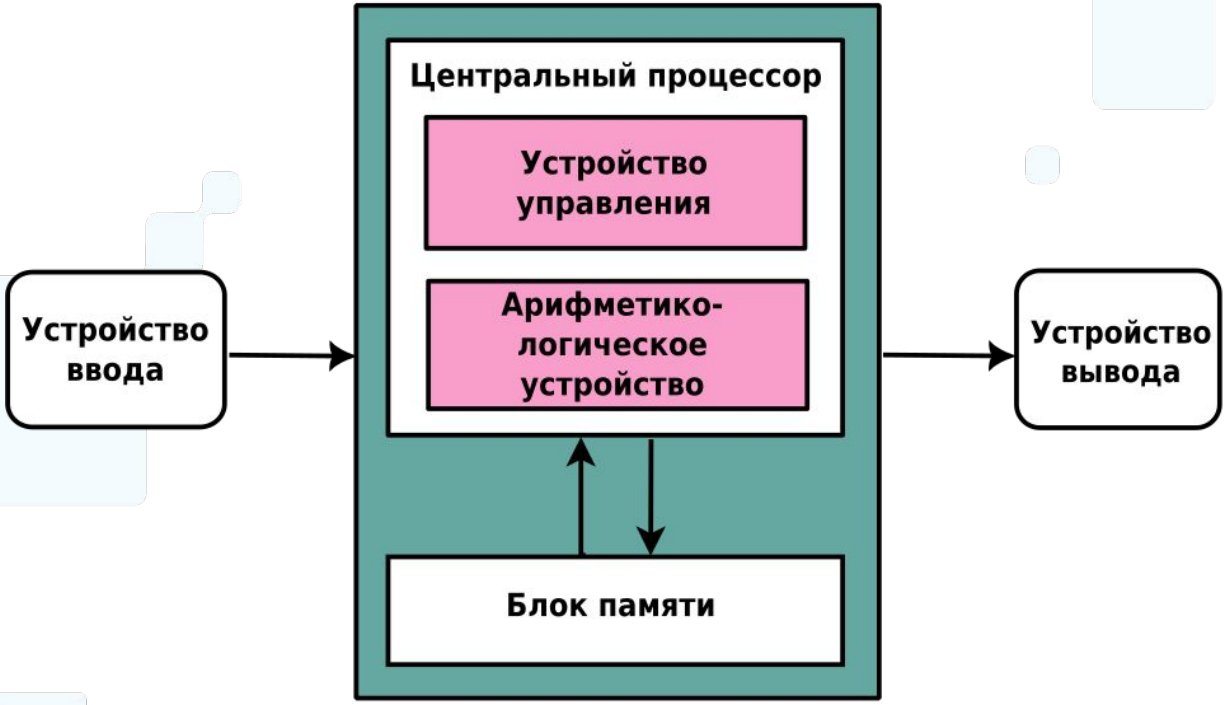
Лекция 6
ОС, IO

Операционная система



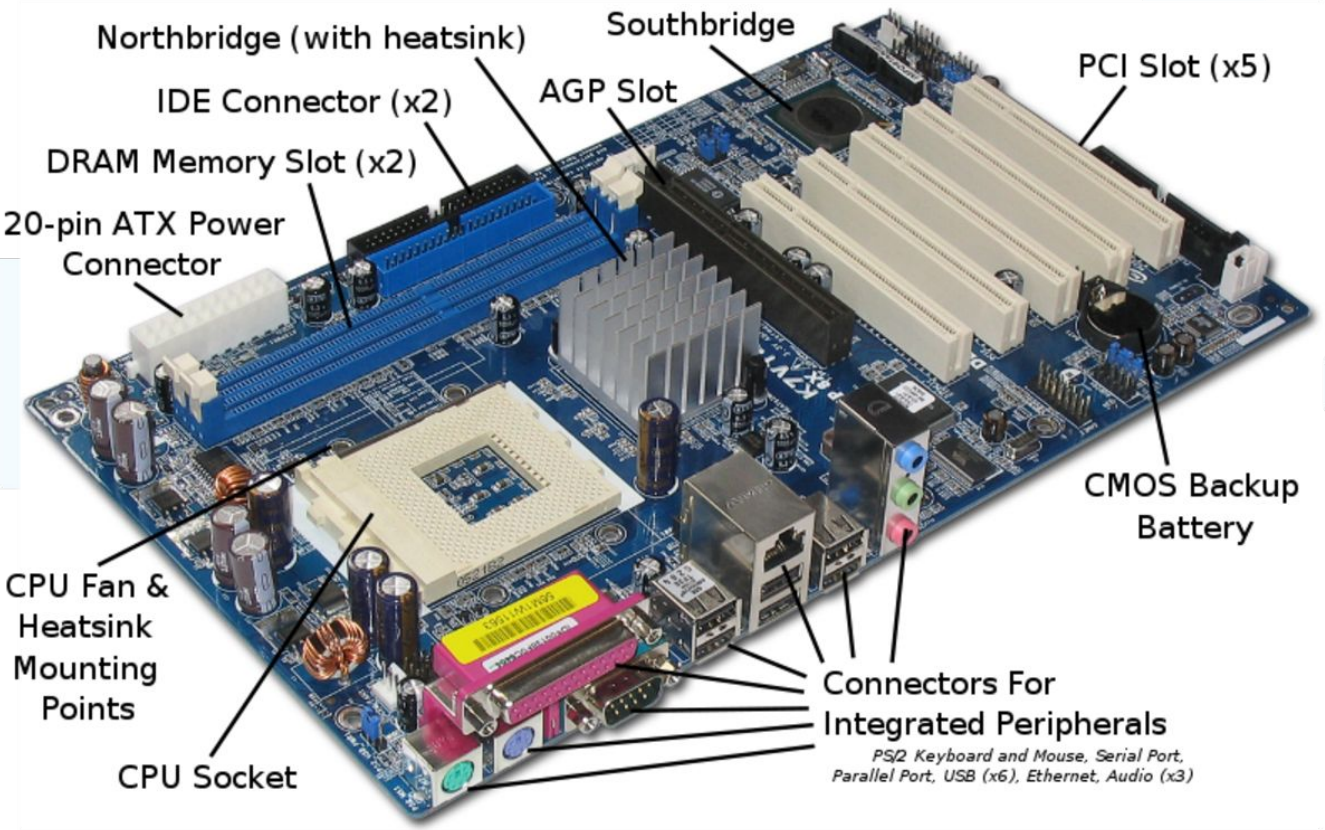
- Ест ресурсы
- Хочет обновляться не вовремя
- Заставляет спорить за то какая лучше
- Не даёт наслаждаться игрой
- Запускает в фоне мильон задач которые не нужны

Архитектура Фон-Неймана

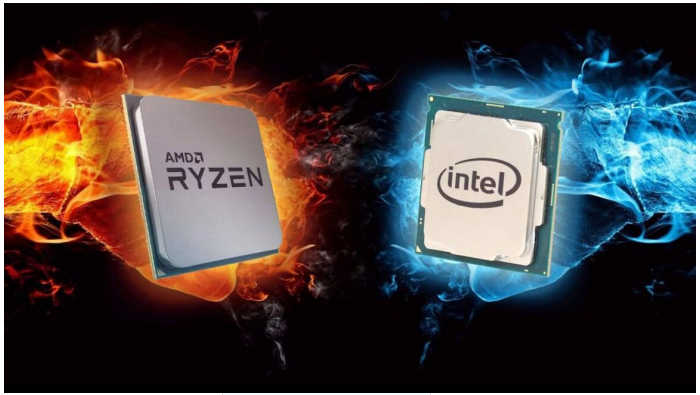


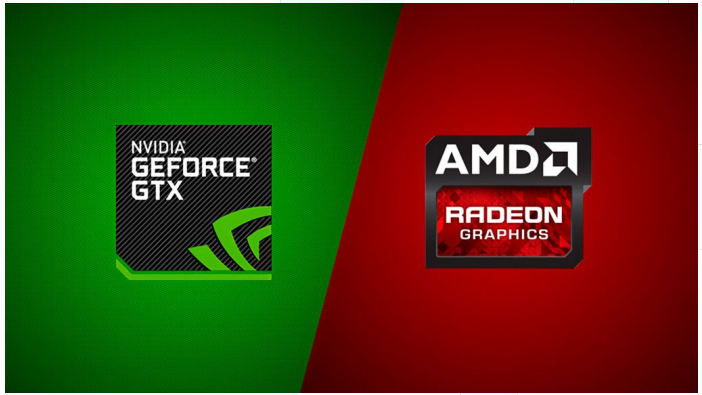
FAST FORWARD

А на самом деле



А на самом деле





HDD

SSD

А на самом деле

Операционная система

Унифицирует (драйверы, файловая система)

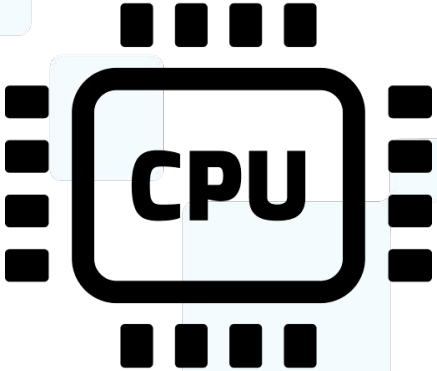
FAST FORWARD

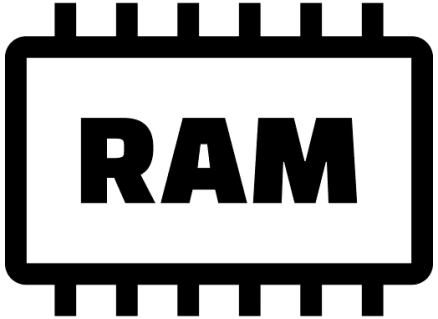
Процессы

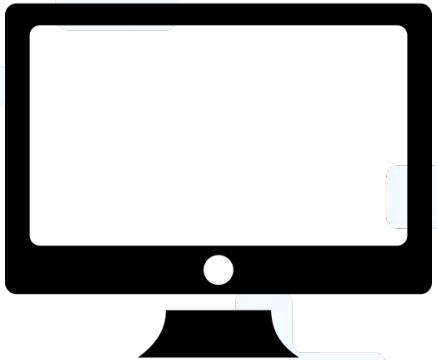
```
  0[|          0.7%]   4[          0.0%]   8[||         2.0%] 12[|          0.7%]
  1[|          0.7%]   5[          0.0%]   9[          0.0%] 13[          0.0%]
  2[           0.0%]   6[|         0.7%]  10[|         0.7%] 14[          0.0%]
  3[           0.0%]   7[          0.0%]  11[          0.0%] 15[          0.0%]
Mem[||||||||||||||||||||||||18.0G/46.8G] Tasks: 104, 622 thr, 283 kthr; 1 runni
Swp[                          0K/0K] Load average: 0.02 0.08 0.09
                                     Uptime: 12:08:15

Main   I/O
  PID USER      PRI  NI  VIRT   RES   SHR S CPU%▽MEM%   TIME+  Command
 1804 root       20   0 4162M 94792 54052 S  0.7  0.2  0:06.26 /usr/bin/dock
 2950 root       20   0 64.9G 11.4G 9162M S  0.7 24.5  2:32.63 VLLM::EngineC
 2960 root       20   0 64.9G 11.4G 9162M S  0.7 24.5  2:32.39 VLLM::EngineC
 2963 root       20   0 64.9G 11.4G 9162M S  0.7 24.5  0:17.34 VLLM::EngineC
 2965 root       20   0 64.9G 11.4G 9162M S  0.7 24.5  2:32.49 VLLM::EngineC
 2970 root       20   0 64.9G 11.4G 9162M S  0.7 24.5  2:32.51 VLLM::EngineC
140130 igr       20   0 10380  6760  3712 R  0.7  0.0  0:01.48 htop
    1 root       20   0 22660 13944  9564 S  0.0  0.0  0:22.33 /sbin/init
  484 root       19  -1 83324 34236 32900 S  0.0  0.1  0:03.03 /usr/lib/syst
  520 root       RT   0  282M 27456  8760 S  0.0  0.1  0:01.08 /sbin/multipa
```

FAST FORWARD

Общие ресурсы







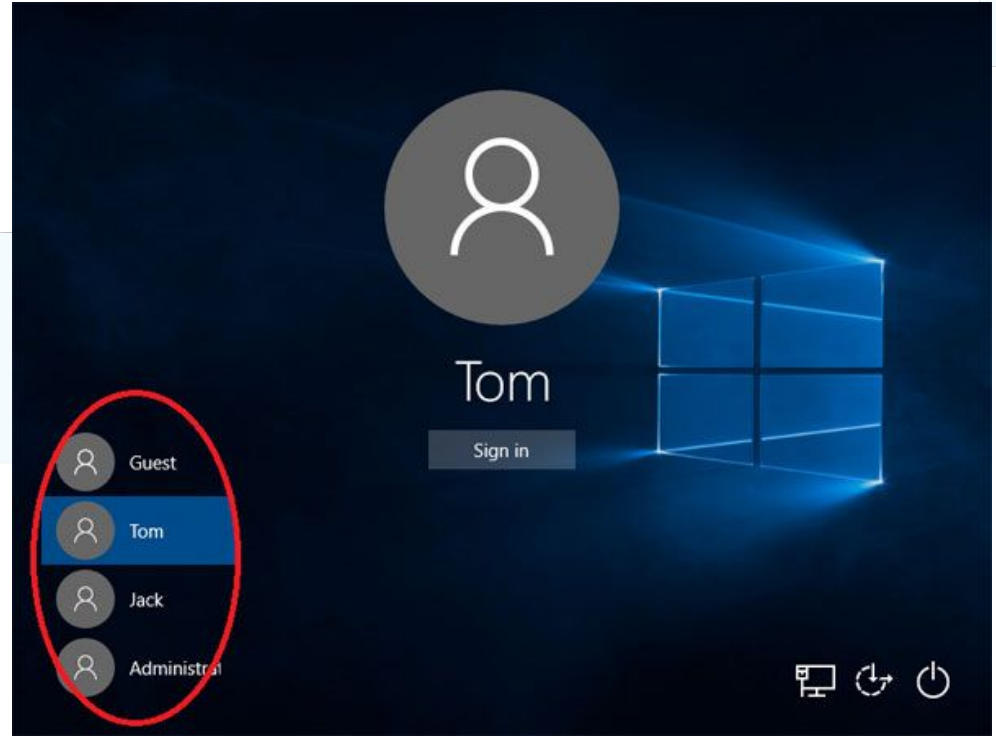
Операционная система

Унифицирует (драйверы, файловая система)

Разделяет (Многозадачность, виртуальная память)

FAST
FORWARD

Пользователи



FAST FORWARD

Операционная система

Унифицирует (драйверы, файловая система)

Разделяет (Многозадачность, виртуальная память)

Разграничивает (Пользователей)

FAST FORWARD

Разные ОС

FAST FORWARD

Разные ОС

FAST FORWARD

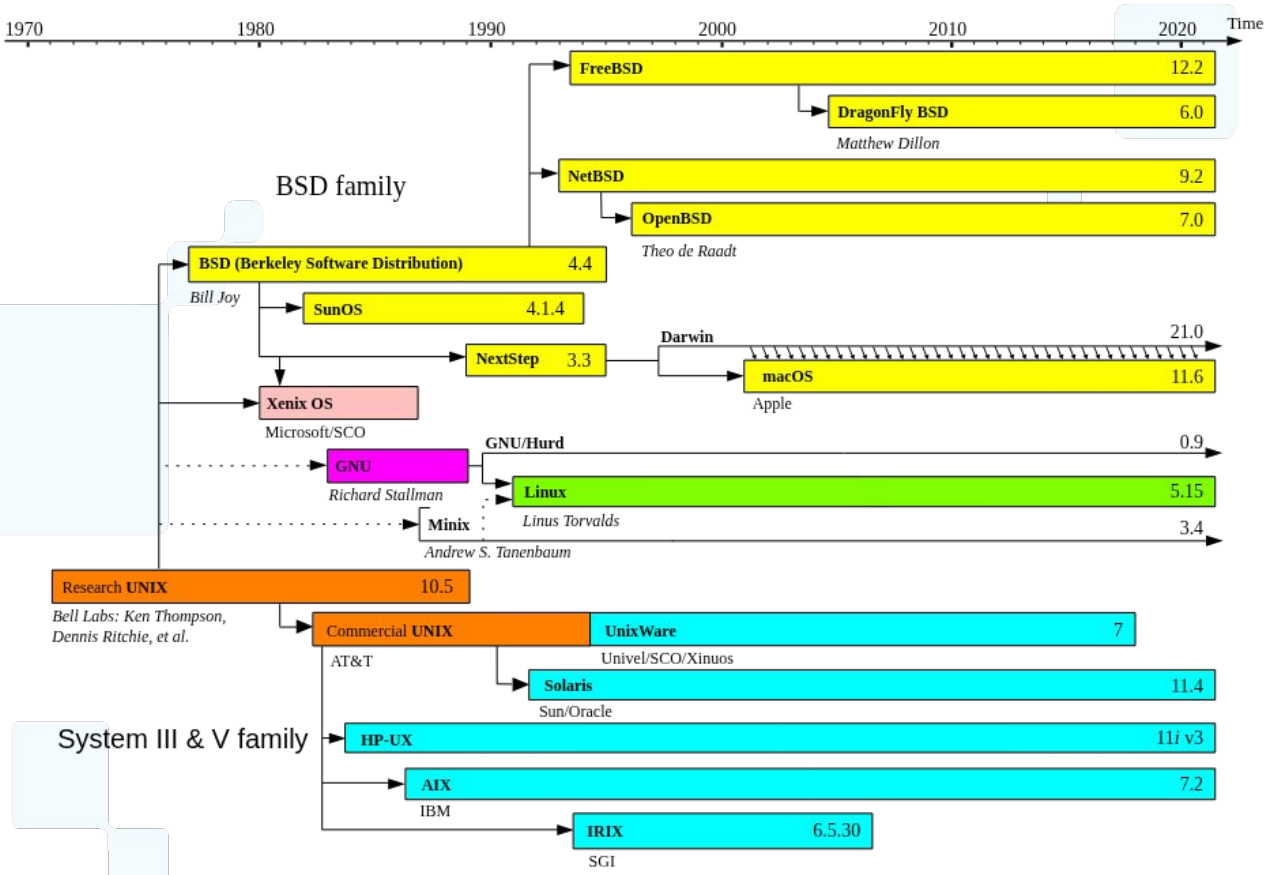
API vs ABI

API (application programming interface) — контракт на уровне исходного кода (source-level).

ABI (application binary interface) — контракт на уровне машинного кода и двоичного представления данных: соглашения о вызовах, способы компоновки, порядок загрузки и запуска программы, интерфейс взаимодействия с ядром ОС.

FAST FORWARD

UNIX



FAST FORWARD

Операционная система

Унифицирует (драйверы, файловая система)

Разделяет (Многозадачность, виртуальная память)

Разграничивает (Пользователей)

FAST FORWARD

Поддержка на уровне железа

Привилегированные инструкции

Переход между kernel space и user space

Взаимодействие с устройствами

Виртуализация

X86. Кольца защиты

Ring 0:

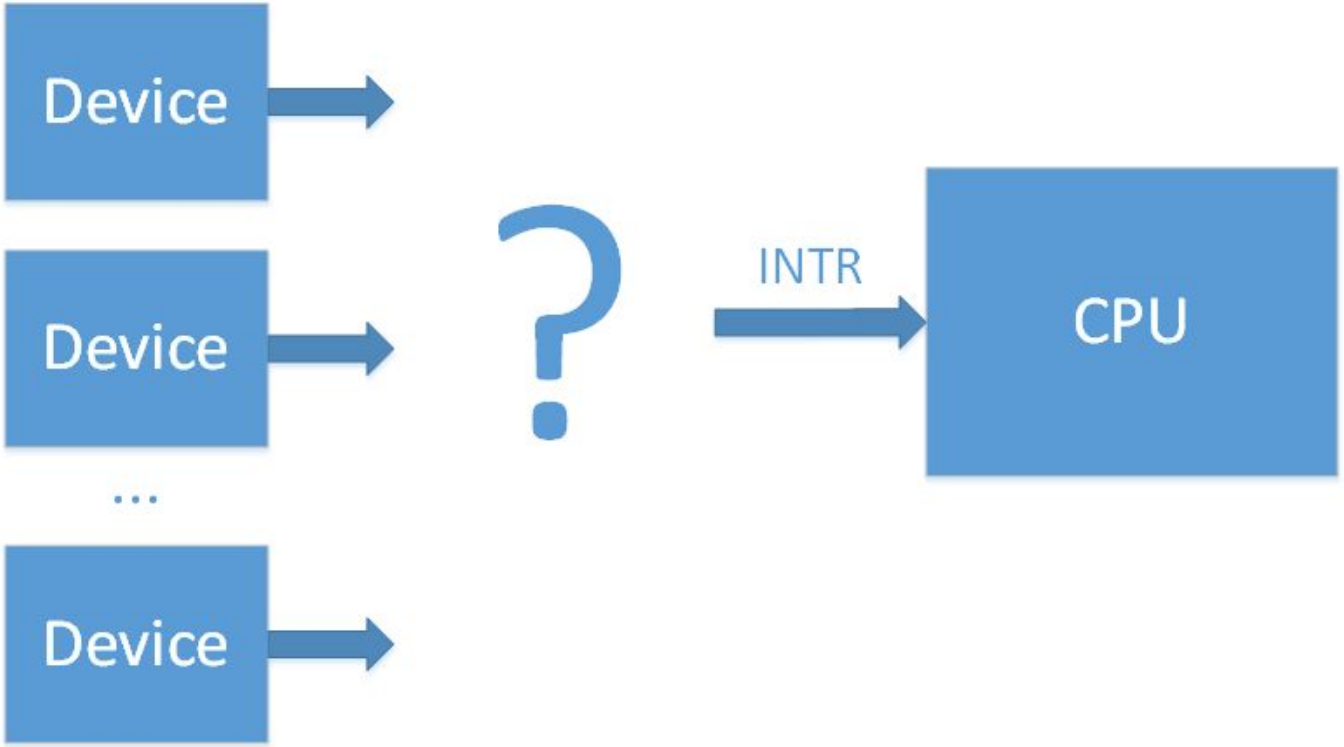
- Доступ к железу (port mapped IO, memory mapped IO)
- Служебные регистры (GDTR, IDTR, контрольные регистры)
- Служебные инструкции (lgdt, lidt, cli/sti, …)

Ring 3:

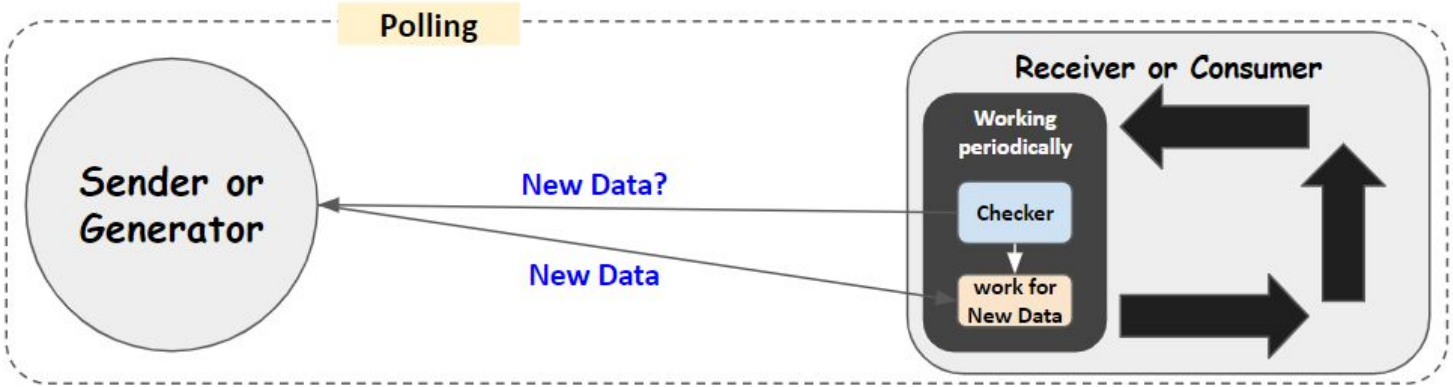
- Доступ к инструкциям/регистрам общего назначения

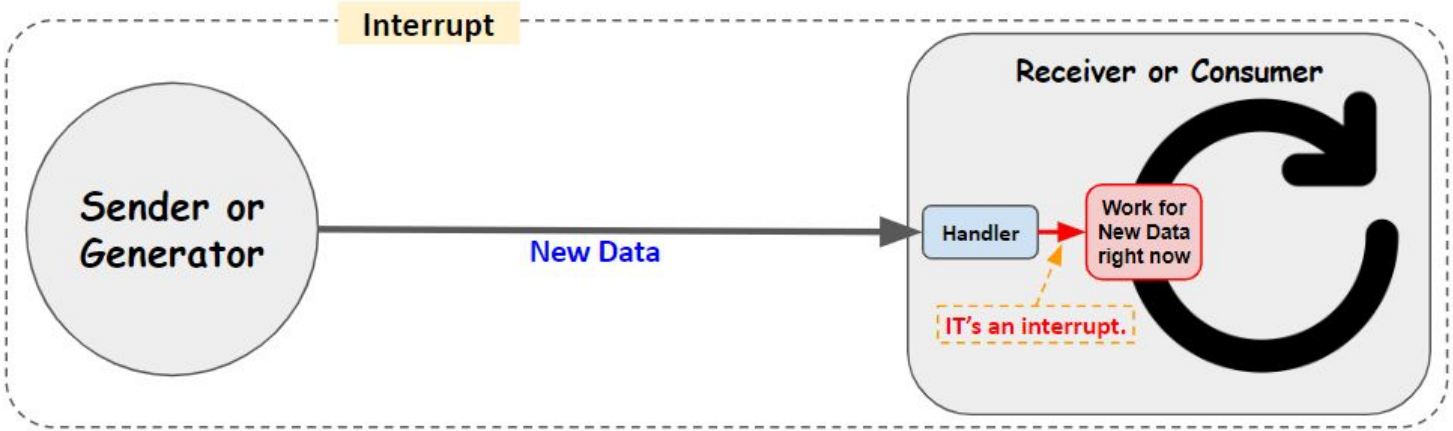
Актуальный уровень привилегий — CPL (current privilege level) — хранится в младших двух битах регистра cs.

X86. Прерывания



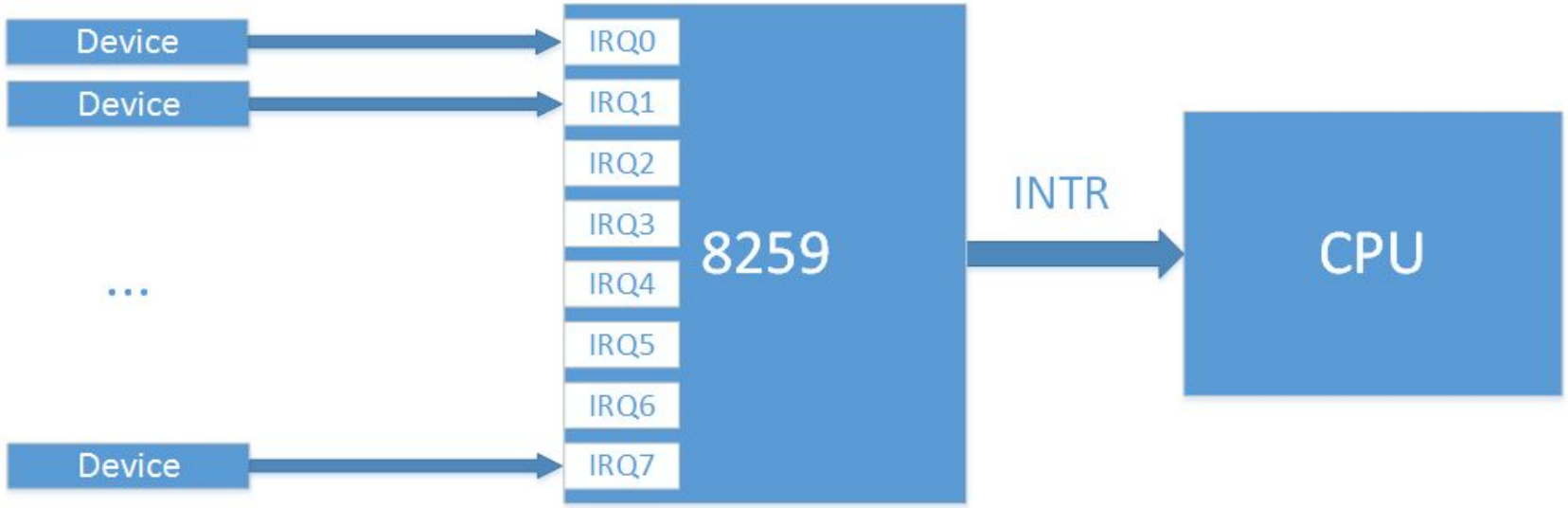
X86. Прерывания



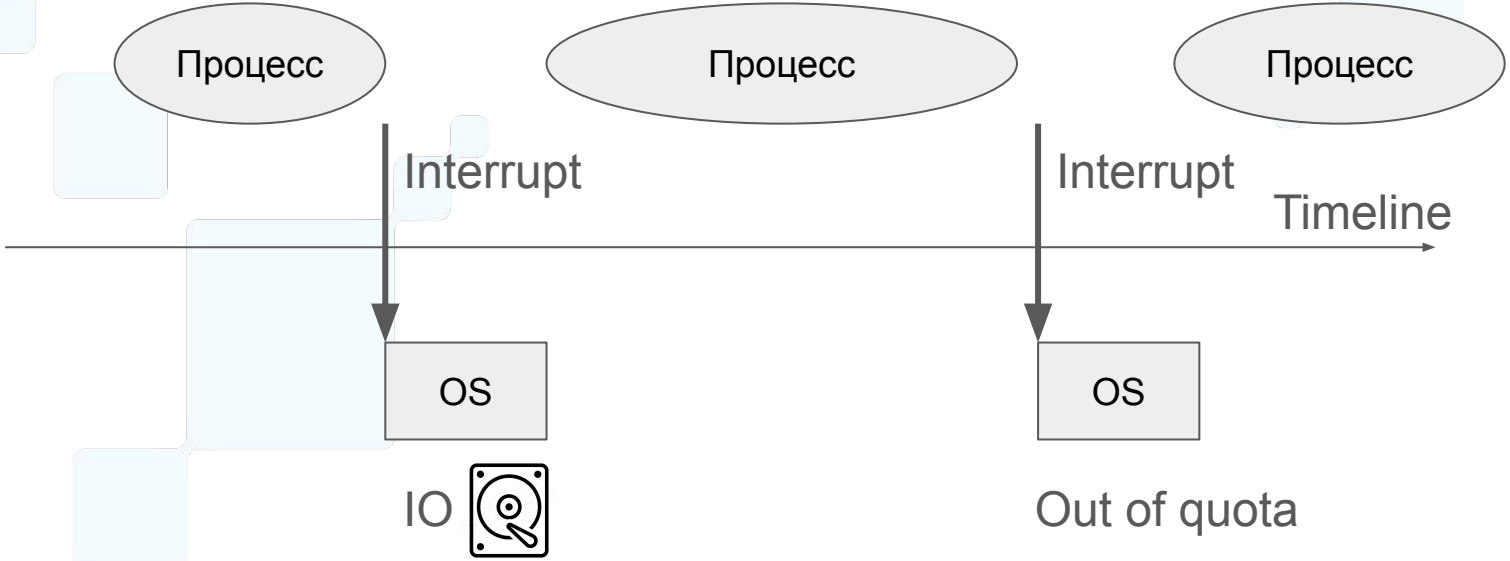


FAST FORWARD

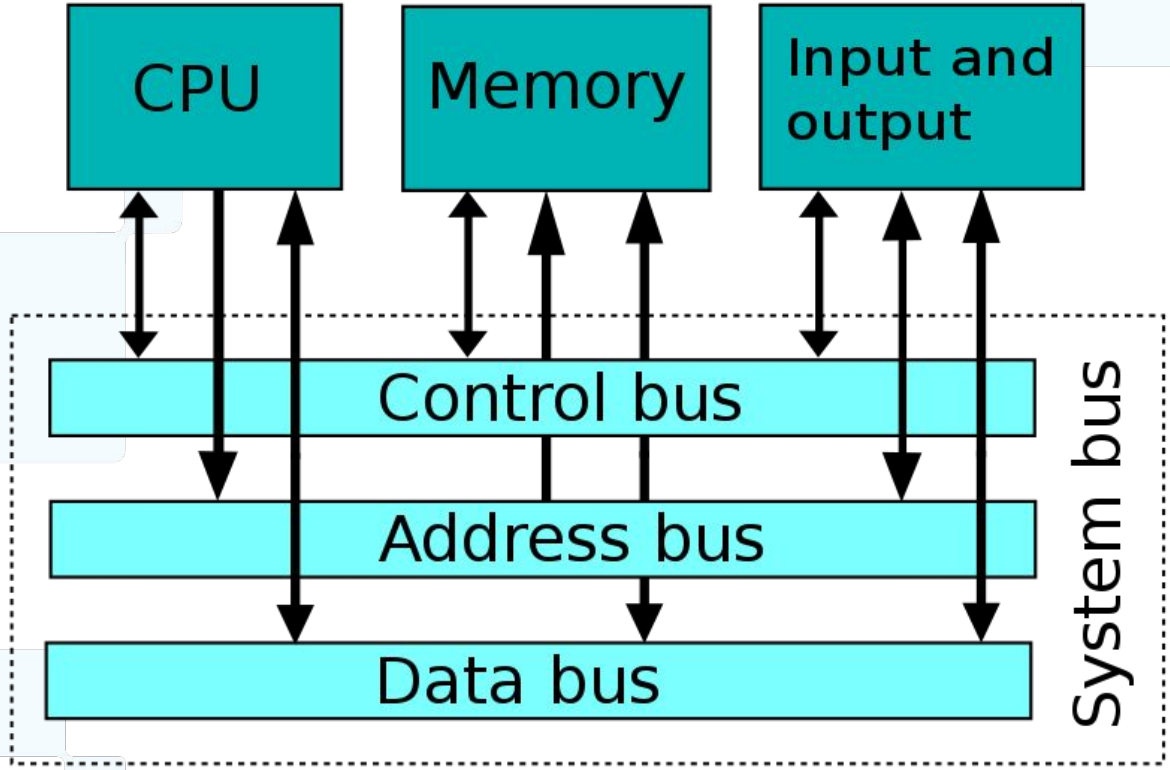
X86. Прерывания



X86. Прерывания



IO


CPU
Memory
Input and output
Control bus
Address bus
Data bus
System bus

FAST FORWARD

Port-mapped IO

IN — Input From Port

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
E4 ib	IN AL, imm8	I	Valid	Valid	Input byte from imm8 I/O port address into AL.
E5 ib	IN AX, imm8	I	Valid	Valid	Input word from imm8 I/O port address into AX.
E5 ib	IN EAX, imm8	I	Valid	Valid	Input dword from imm8 I/O port address into EAX.
EC	IN AL,DX	ZO	Valid	Valid	Input byte from I/O port in DX into AL.
ED	IN AX,DX	ZO	Valid	Valid	Input word from I/O port in DX into AX.
ED	IN EAX,DX	ZO	Valid	Valid	Input doubleword from I/O port in DX into EAX.

Instruction Operand Encoding ¶

Op/En	Operand 1	Operand 2	Operand 3	Operand 4
I	imm8	N/A	N/A	N/A
ZO	N/A	N/A	N/A	N/A

FAST FORWARD

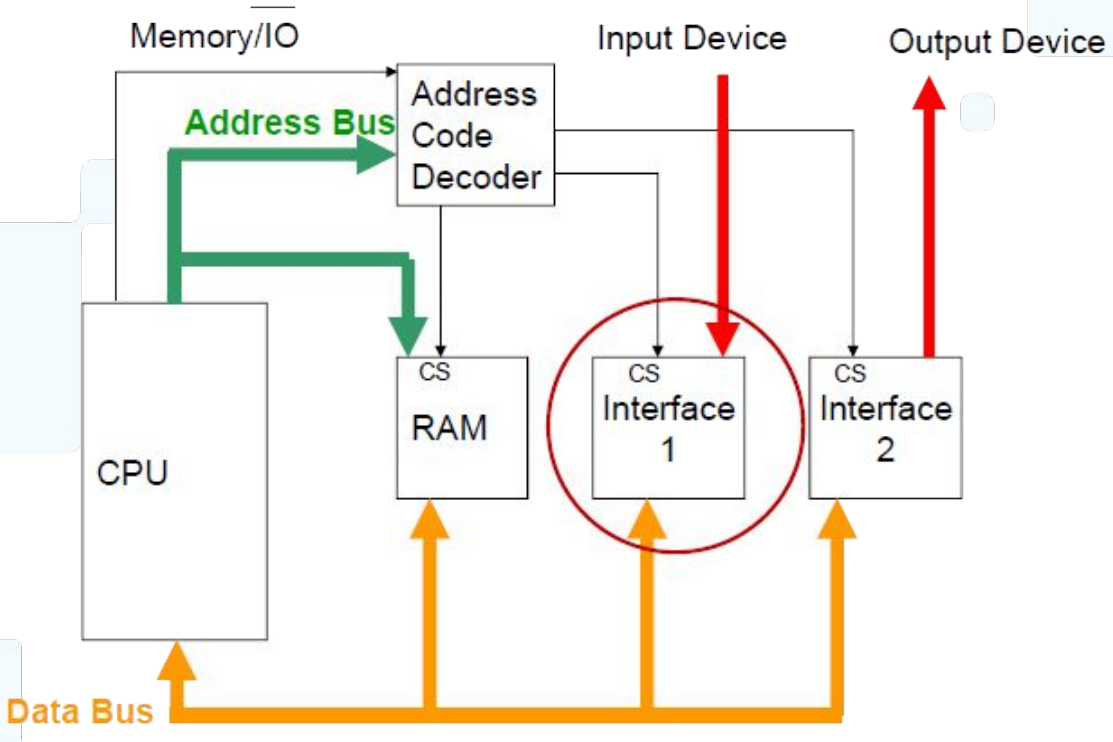
Port-mapped IO

OUT — Output to Port

Opcode[1]

	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
E6 ib	OUT imm8, AL	I	Valid	Valid	Output byte in AL to I/O port address imm8.
E7 ib	OUT imm8, AX	I	Valid	Valid	Output word in AX to I/O port address imm8.
E7 ib	OUT imm8, EAX	I	Valid	Valid	Output doubleword in EAX to I/O port address imm8.
EE	OUT DX, AL	ZO	Valid	Valid	Output byte in AL to I/O port address in DX.
EF	OUT DX, AX	ZO	Valid	Valid	Output word in AX to I/O port address in DX.
EF	OUT DX, EAX	ZO	Valid	Valid	Output doubleword in EAX to I/O port address in DX.

Memory-mapped IO



FAST FORWARD

Системные вызовы

SYSCALL — Fast System Call

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
0F 05	SYSCALL	ZO	Valid	Invalid	Fast call to privilege level 0 system procedures.

Instruction Operand Encoding ¶

Op/En	Operand 1	Operand 2	Operand 3	Operand 4
ZO	N/A	N/A	N/A	N/A

Системные вызовы linux x86_64

```
long long result, arg1, arg2, arg3;
asm volatile("syscall"
    : "=a"(result)                          // output
    : "a"(SYS_...),                         // input: syscall number (in eax/rax)
      "D"(arg1), "S"(arg2), "d"(arg3)  // inputs: arguments (rdi, rsi, rdx)
    : "rcx", "r11", "memory")
```

FAST FORWARD

Операционная система

Унифицирует (драйверы, файловая система)

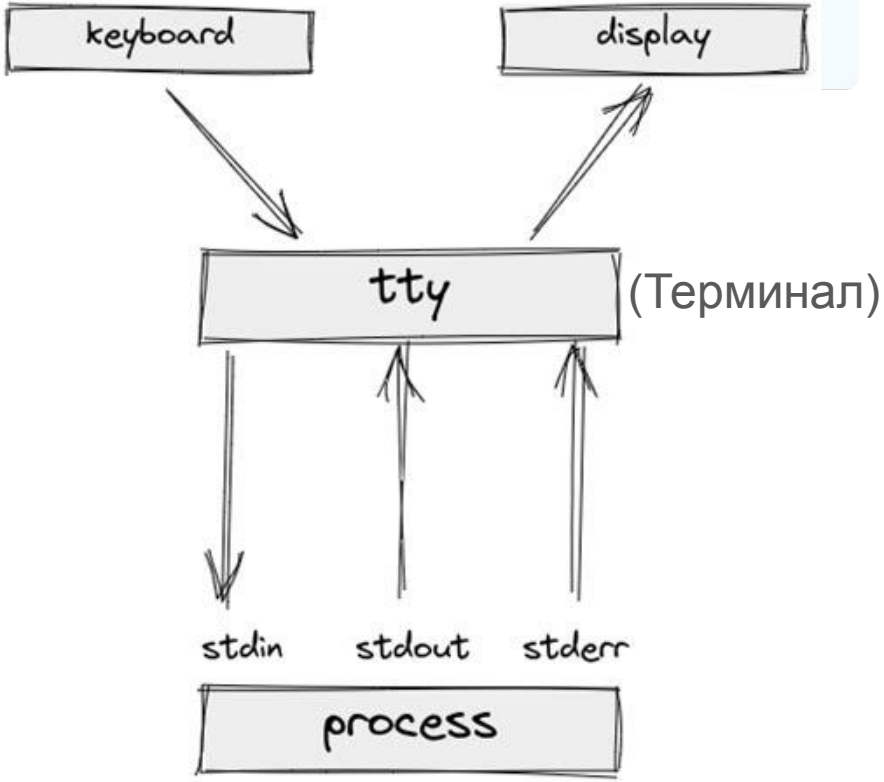
Разделяет (Многозадачность, виртуальная память)

Разграничивает (Пользователей)

Ввод вывод

$ gcc … -o myprog

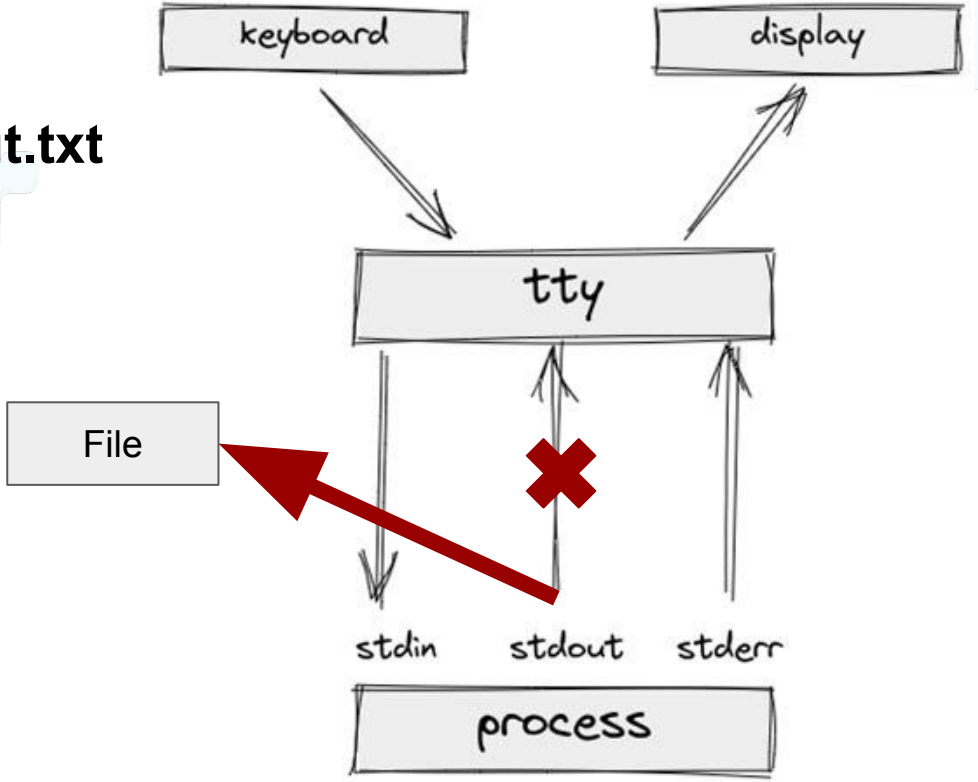
$ ./myprog



Ввод вывод

$ gcc … -o myprog

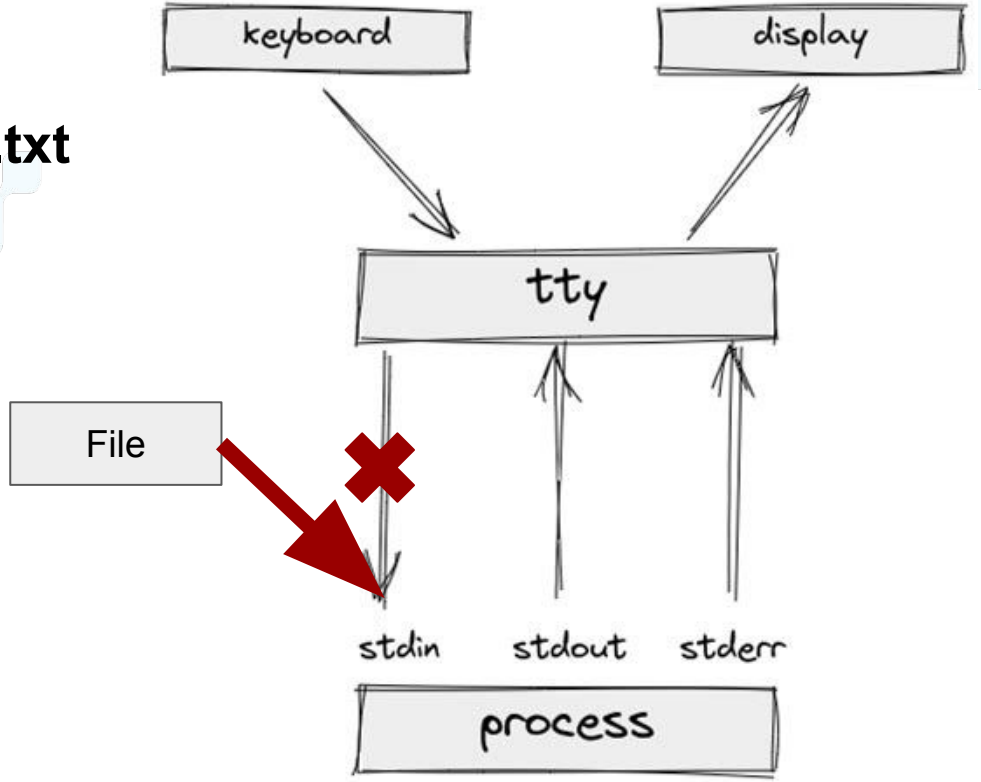
$ ./myprog **> output.txt**



Ввод вывод

$ gcc … -o myprog

$ ./myprog **< input.txt**



Файл

Файл - последовательность байт с доступом через файловые операции (системные вызовы)

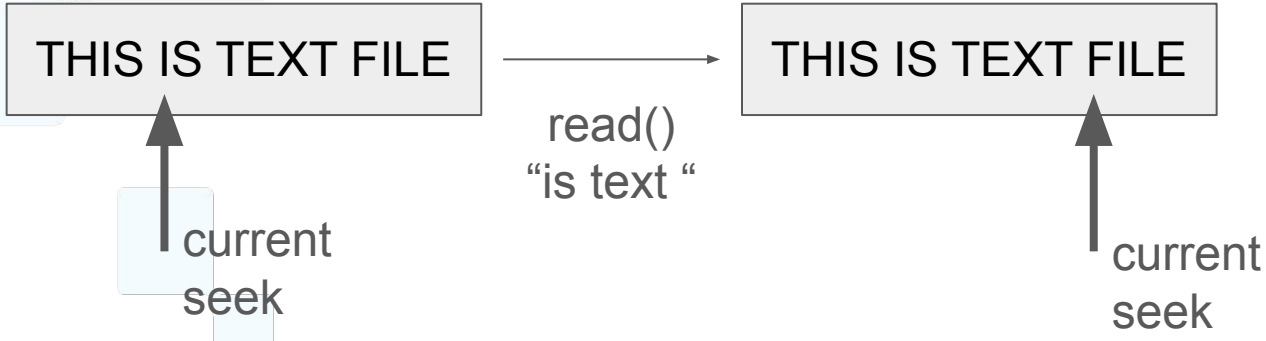
- файл на диске
- вывод программы
- пользовательский ввод
- бесконечная последовательность случайных байт
- всё содержимое диска
- и.т.д.

Операции

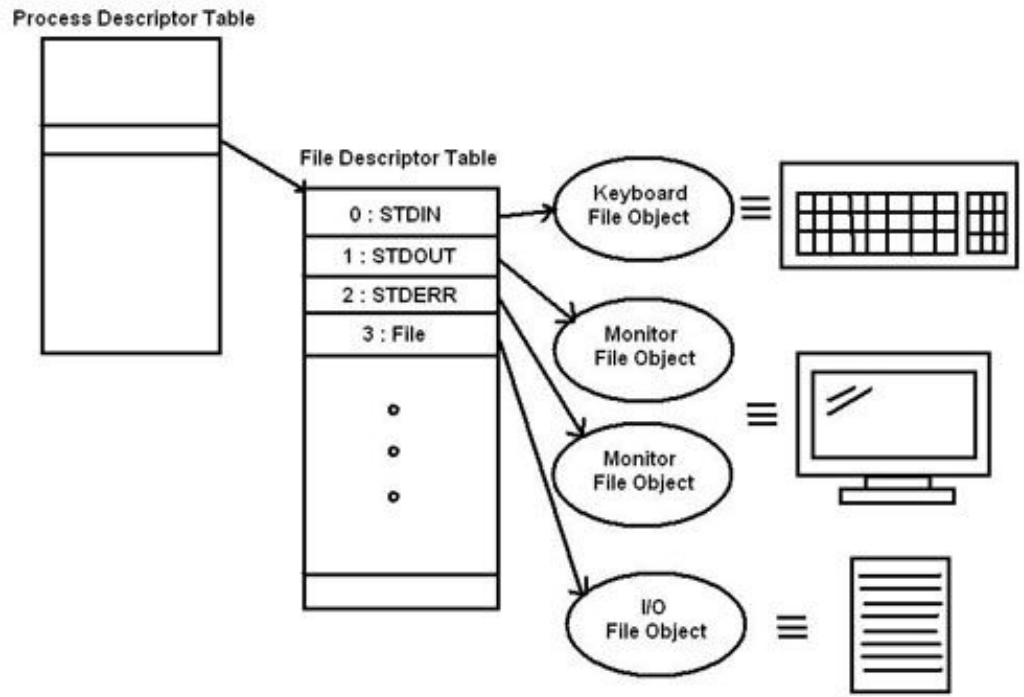
Read - прочитать

Write - записать

Seek - поменять местоположение чтения/записи



Файловые дескрипторы



- 0 (STDIN_FILENO) — standard input
- 1 (STDOUT_FILENO) — standard output
- 2 (STDERR_FILENO) — standard error output

POSIX API

```
ssize_t read(int fd, void *buf, size_t count);

ssize_t write(int fd, const void *buf, size_t count);
```

- Чтение/запись с текущей позиции вперед
- Не гарантируют что используют весь буфер
- На ошибке вернут **-1** и в **errno** код
- ssize_t - знаковый size_t
- read вернет 0 если **EOF**

FAST FORWARD

Пример - cat

```
#include <unistd.h>

int main() {
    char c;  // буфер в один байт
    while (read(STDIN_FILENO, &c, sizeof(c)) > 0) {
        write(STDOUT_FILENO, &c, sizeof(c));
    }
}
```

POSIX API

```
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
```

- Успех – возвращает номер дескриптора
- Ошибка – -1 и errno
- pathname – путь
- O_RDONLY, O_WRONLY и O_RDWR – режимы доступа
- O_TRUNC обрезает файл
- O_CREAT создаст файл, если его нет
- O_APPEND установит seek на конец файла
- mode – права доступа к файлу

FAST FORWARD

Пример - cat

```
int main(int argc, char *argv[]) {
    char c;  // буфер в один байт
    int fd = open(argv[1], O_RDONLY);
    while (read(fd, &c, sizeof(c)) > 0) {
        write(STDOUT_FILENO, &c, sizeof(c));
    }
}
```

FAST FORWARD

Пример - cat

```
int main(int argc, char *argv[]) {
    char c;  // буфер в один байт
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0666);
    while (read(STDIN_FILENO, &c, sizeof(c)) > 0) {
        write(fd, &c, sizeof(c));
    }
}
```

POSIX API

```
int close(int fd);
```

- Утечка ресурсов - номер в таблице
- Всего по дефолту 1024 элемента
- Но при выходе из программы все закрывается само

POSIX API

```
off_t lseek(int fd, off_t offset, int whence);
```

- offset - количество байт
- whence - точка отсчета
    - SEEK_SET - с начала файла
    - SEEK_CUR - с текущей позиции
    - SEEK_END - с конца файла

FAST FORWARD

Пример - file size

```
off_t get_file_size(const char * path) {
    int fd = open(path, O_RDONLY);
    off_t file_size = lseek(fd, 0, SEEK_END);
    close(fd);
    return file_size;
}
```

FAST FORWARD

А в СИ?

```
FILE * fopen(const char *pathname, const char *mode);
int fclose(FILE *stream);
int fprintf(FILE *stream, const char *format, ...);
int fscanf(FILE *stream, const char *format, ...);
int fseek(FILE *stream, long offset, int whence);
```

А в СИ?

```
FILE * fopen(const char *pathname, const char *mode);
```

r Чтение
r+ Чтение и запись
w Запись. Создать и очистить файл
w+ Чтение и запись. Создать и очистить файл
a Добавление (запись, но seek на конец файла)
a+ Добавление и чтение.

Права при создании - 0666

Ошибка - NULL

FAST FORWARD

Пример

```
int main() {
    FILE *f = fopen("poem. txt", "w");
    fprintf(f, "some string");
    fprintf(f, " and some other string\n");
    pause();
    fclose(f);
}
```

FAST FORWARD

А в СИ?

```
FILE * fopen(const char *pathname, const char *mode);
int fclose(FILE *stream);
int fprintf(FILE *stream, const char *format, ...);
int fscanf(FILE *stream, const char *format, ...);
int fseek(FILE *stream, long offset, int whence);

int fflush(FILE *stream);
```

FAST FORWARD

Пример

```
int main() {
    FILE *f = fopen("poem. txt", "w");
    fprintf(f, "some string");
    fprintf(f, " and some other string\n");
    fflushf(f);
    pause();
    fclose(f);
}
```

FAST FORWARD

stdin stdout

```
int fprintf(FILE *stream, const char *format, ...);
int fscanf(FILE *stream, const char *format, ...);
```

stdin, stdout, stderr

printf(...) <-> fprintf(stdout, …)

scanf(...) <-> fscanf(stdin, …)

stderr - без буфера

FAST FORWARD

exit vs _exit

```
void exit(int status);    // сбрасываем буфер
void _exit(int status);   // завершаем без сброса
```

Нюансы

```
void setbuf(FILE *stream, char *buf); // для управления буферизацией
void setlinebuf(FILE *stream);        // для строчной буферизации
```

Перенос строки UNIX – \n

Перенос строки WINDOWS – \r \n

Отключить – при открытии - mode "b"

FAST FORWARD

АКОС

QCourse.ru 2026